

Vision Dual-Arm Teleoperation — Complete Project Guide (Version 1)

This document explains the **main aim**, **architecture**, **mathematics**, and **every major component** so you can answer questions about this project confidently.

1. Main Aim

Vision-Based Dual-Arm Teleoperation and Demonstration Collection for Robotic Manipulation

The project turns **human hand motion captured by a webcam** into **robot arm control commands**, with the long-term goal of recording demonstrations for **imitation learning (IL)** or **reinforcement learning (RL)**.

MVP (Minimum Viable Product)

Webcam → Hand tracking → Gesture detection → Velocity commands → Gripper open/close → CSV recording

Final Demo Goal

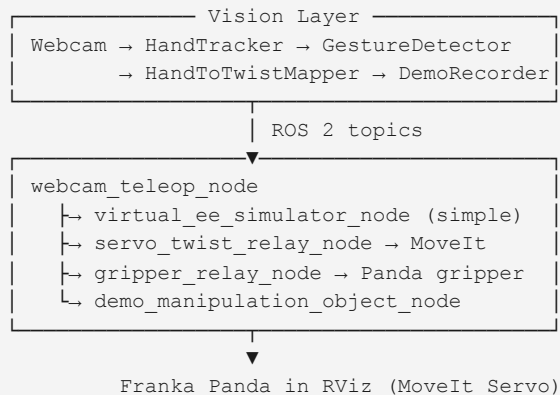
A video showing:

1. Real-time hand tracking
2. Pinch gesture detection
3. Hand motion mapped to end-effector velocity
4. Simulated robot arm moving (Panda via MoveIt Servo)
5. Pick-and-place attempt
6. Demonstration data saved to CSV

Why This Architecture?

Choice	Reason
MediaPipe Hands	Fast, no GPU required, 21 landmarks per hand
Velocity (twist) control	Natural for continuous teleoperation; MoveIt Servo accepts TwistStamped
ROS 2 Jazzy	Industry-standard middleware; integrates with MoveIt 2
Franka Panda first	Well-documented MoveIt config; good for gripper demos
MoveIt Servo over Isaac Sim	Faster setup for a 1-week demo; collision checking built-in

2. High-Level Architecture



3. Repository Structure

```

vision_dual_arm_teleop/
├── vision_dual_arm_teleop/      # Core Python library (vision + mapping)
│   ├── tracking/
│   │   ├── hand_tracker.py      # MediaPipe wrapper
│   │   └── gesture_detector.py  # Pinch/open detection
│   ├── mapping/
│   │   └── hand_to_twist.py      # Hand position → robot velocity
│   └── recording/
│       └── demo_recorder.py      # CSV demo logging
├── ros2_ws/src/vdat_teleop/     # ROS 2 package
│   └── vdat_teleop/
│       ├── webcam_teleop_node.py
│       ├── virtual_ee_simulator_node.py
│       ├── servo_twist_relay_node.py
│       ├── gripper_relay_node.py
│       ├── scene_objects_node.py
│       └── demo_manipulation_object_node.py
├── scripts/run_hand_tracking.py  # Standalone (no ROS)
└── data/demos/                  # Recorded CSV files
  
```

Important: The core library lives outside the ROS package. ROS nodes import it via PYTHONPATH:

```
export PYTHONPATH=/home/shubham.ghogare/vision_dual_arm_teleop:$PYTHONPATH
```

4. Vision Layer — How Hand Tracking Works

4.1 MediaPipe Hands

MediaPipe detects up to **2 hands** and outputs **21 3D landmarks** per hand in **normalized image coordinates**:

- x, y in [0, 1] — fraction of image width/height

- z — depth relative to wrist (not metric depth)

Key landmark indices used in this project:

Index	Landmark
0	Wrist
4	Thumb tip
8	Index tip
12	Middle tip
16	Ring tip
20	Pinky tip

The `HandTracker` class wraps MediaPipe. Each frame: convert BGR→RGB, run MediaPipe, extract wrist (landmark 0) and index tip (landmark 8), run gesture detection, and label hands as "Left" or "Right".

Note: The frame is **horizontally flipped** (`cv2.flip(frame, 1)`) so movement feels mirror-natural to the operator.

Currently only the right hand is used for teleoperation (`observations["Right"]`).

4.2 Gesture Detection — Pinch Math

Pinch is detected by **Euclidean distance** between thumb tip (landmark 4) and index tip (landmark 8):

```
d_pinch = sqrt((x4 - x8)^2 + (y4 - y8)^2 + (z4 - z8)^2)
```

Pinch condition: `is_pinching = True` if `d_pinch < 0.055` (normalized coordinate units).

An additional metric, `hand_open_score`, is the average wrist-to-fingertip distance across index/middle/ring/pinky — useful for future open-hand detection but not currently used for control.

5. Mapping Layer — Hand Position → Robot Velocity

This is the core **teleoperation mapping**. The design uses **velocity control** (not absolute position mapping), which is more robust to calibration errors.

5.1 Coordinate Conventions

Camera/image frame:

- x increases to the **right**
- y increases **downward**
- Origin at top-left corner

Robot command frame (MVP):

- Hand left/right → robot $\pm v_y$ (lateral)
- Hand up/down → robot $\pm v_z$ (vertical)
- Depth (v_x) → keyboard W/S (not yet from hand size)

5.2 Low-Pass Filter (Smoothing)

Raw wrist positions are noisy. An exponential moving average filter is applied:

```
x_filt(t) = alpha * x_raw(t) + (1 - alpha) * x_filt(t-1)
```

With $\alpha = 0.25$, this is a **first-order low-pass filter**. Lower α = smoother but more lag.

5.3 Deadband

Small hand movements near center should not move the robot:

```
deadband(v) = 0    if |v| < 0.06
              = v    otherwise
```

Applied after centering:

```
dx = deadband(x_wrist - 0.5)
dy = deadband(0.5 - y_wrist)
```

Note: y is inverted (0.5 - y) because image y goes down but robot z goes up.

5.4 Velocity Scaling and Clipping

```
vy = clip(dx * v_max * 2, -v_max, v_max)
vz = clip(dy * v_max * 2, -v_max, v_max)
```

Default $v_max = 0.20$ m/s. The factor of 2 maps full hand displacement from center (0.5) to max speed at image edges.

5.5 Output: TeleopCommand

```
TeleopCommand:
  vx: float    # forward/back (keyboard W/S in ROS node)
  vy: float    # left/right (from hand)
  vz: float    # up/down (from hand)
  gripper_close: bool # from pinch gesture
```

6. Demonstration Recording

`DemoRecorder` logs every frame to CSV for future IL/RL training:

Column	Description
time	Unix timestamp
handedness	"Right" or "Left"
wrist_x, wrist_y	Normalized wrist position
pinch_distance, is_pinching	Gesture state
cmd_vx, cmd_vy, cmd_vz	Output velocity commands
gripper_close	0 or 1

Run with: `python scripts/run_hand_tracking.py --record`

7. ROS 2 Layer — Nodes and Topics

7.1 Topic Map

Topic	Type	Publisher	Subscriber(s)
/teleop/right_arm/twist_cmd	TwistStamped	webcam_teleop_node	virtual_ee_simulator_node, servo_twist_relay_node
/teleop/right_gripper/close	Bool	webcam_teleop_node	gripper_relay_node, demo_manipulation_object_node
/teleop/right_arm/ee_pose	PoseStamped	virtual_ee_simulator_node	RViz
/teleop/right_arm/ee_marker	Marker	virtual_ee_simulator_node	RViz
/servo_node/delta_twist_cmds	TwistStamped	servo_twist_relay_node	Movelt Servo
/panda_hand_controller/joint_trajectory	JointTrajectory	gripper_relay_node	Panda gripper controller
/planning_scene	PlanningScene	scene_objects_node, demo_manipulation_object_node	Movelt
/teleop/scene_markers	MarkerArray	scene_objects_node	RViz
/teleop/manipulation_object_markers	MarkerArray	demo_manipulation_object_node	RViz

7.2 webcam_teleop_node — Main Teleop Publisher

Runs at **30 Hz**. Each timer callback:

1. Reads webcam frame (flipped)
2. Runs HandTracker.process()
3. If **no right hand** → publishes zero velocity + gripper open (safety stop)
4. Maps wrist → TeleopCommand via HandToTwistMapper
5. Adds keyboard depth: **W** = +vx, **S** = -vx (default 0.12 m/s)
6. Publishes TwistStamped and Bool gripper message
7. Shows OpenCV GUI with overlay

Safety: Hand loss immediately stops the robot.

7.3 virtual_ee_simulator_node — Simple 3D Simulator

Integrates velocity into position using **Euler integration**:

$$p(t + dt) = p(t) + v * dt$$

Where $p = [x, y, z]$, $v = [vx, vy, vz]$.

Workspace clipping keeps the virtual gripper inside bounds:

Axis	Min	Max
x	0.20	0.75
y	-0.45	0.45
z	0.10	0.75

Initial position: (0.45, 0.0, 0.35) meters in base_link/map frame.

Publishes a green sphere marker for RViz visualization — **no real robot kinematics**.

7.4 servo_twist_relay_node — Bridge to MoveIt Servo

Pipeline: Raw twist → Scale → Speed limit → Timeout check → Low-pass → Accel limit → Publish

1. Scaling: $v_{raw} = scale * v_{input}$, $scale = 0.35$ (default)

2. Speed vector limiting (preserve direction, cap magnitude):

```
if ||v|| > v_max: v <- v * (v_max / ||v||)
```

Default $v_{max} = 0.08$ m/s.

3. Deadman timeout: If no input for 0.25 s → target velocity = 0.

4. Low-pass smoothing ($\alpha = 0.25$).

5. Acceleration limiting (per axis):

```
delta_v_max = a_max * dt, a_max = 0.25 m/s^2
v_out = v_prev + clip(v_target - v_prev, -delta_v_max, delta_v_max)
```

Output frame: panda_link0 (Panda base).

7.5 gripper_relay_node — Pinch → Panda Gripper

Subscribes to /teleop/right_gripper/close and publishes JointTrajectory to /panda_hand_controller/joint_trajectory.

State	Finger joint positions
Open	0.04 m (each finger)
Closed	0.0 m

Only publishes on **state change** (edge-triggered, not continuous).

7.6 scene_objects_node — Static Pick-and-Place Scene

Object	Size (m)	Position (x,y,z)	Collision?
Table	0.80 x 0.60 x 0.04	(0.55, 0.00, -0.04)	Yes
Pick cube	0.05 x 0.05 x 0.05	(0.50, 0.22, 0.04)	Yes
Place zone	0.12 x 0.12 x 0.01	(0.50, -0.22, 0.015)	No (visual only)

Frame: panda_link0.

7.7 demo_manipulation_object_node — Simulated Grasp/Release

Grasp logic (rising edge on pinch + proximity):

```
attach if: not prev_closed AND closed AND not attached AND ||p_gripper - p_object|| <= 0.16 m
```

Release logic (falling edge on pinch):

```
detach if: prev_closed AND not closed AND attached
```

On attach: remove from world, add as AttachedCollisionObject on panda_hand link, object follows gripper via TF.

On detach: remove attached object, re-add to world at current gripper-relative position.

Uses **TF2** to look up panda_link0 → panda_hand transform.

8. Robot Control — MoveIt Servo Mathematics

When connected to the real Panda simulation, MoveIt Servo solves **inverse kinematics at velocity level**.

Given end-effector twist command $v_{ee} = [x_dot, y_dot, z_dot, \omega_x, \omega_y, \omega_z]$, Servo computes joint velocities via the **Jacobian**:

```

$$\dot{q} = J\_dagger(q) * v_{ee}$$

```

Where J is the 6x7 manipulator Jacobian and J_dagger is its pseudo-inverse (with singularity handling).

This project sends **linear velocity only** (angular = 0), so only the first 3 rows of the Jacobian matter for translation.

MoveIt Servo also provides:

- **Collision checking** — stops motion if collision detected
- **Singularity handling** — slows near singular configurations
- **Joint limit enforcement**

Command type must be switched to **Twist** mode:

```
ros2 service call /servo_node/switch_command_type moveit_msgs/srv/ServoCommandType "{command_type: 1}"
```

9. Three Operating Modes

Mode 1: Standalone Hand Tracking (No ROS)

```
python scripts/run_hand_tracking.py [--record] [--camera 1]
```

Tests vision pipeline only. Shows OpenCV window with landmarks, command arrow, gripper state.

Mode 2: Virtual End-Effector (ROS, No Robot)

Terminal 1: `ros2 run vdat_teleop webcam_teleop_node`

Terminal 2: `ros2 run vdat_teleop virtual_ee_simulator_node`

Terminal 3: `rviz2` → add Marker on `/teleop/right_arm/ee_marker`, Fixed Frame = `map`

A green sphere moves in 3D space following your hand.

Mode 3: Full Panda Demo (MoveIt Servo)

Terminal 1: `ros2 launch moveit_servo demo_ros_api.launch.py`

Terminal 2: Switch to twist mode (service call above)

Terminal 3: `ros2 run vdat_teleop webcam_teleop_node`

Terminal 4: `ros2 run vdat_teleop servo_twist_relay_node --ros-args -p scale:=0.35`

Optional: `gripper_relay_node`, `scene_objects_node`, `demo_manipulation_object_node`

10. Control Mapping Summary

Human Action	Robot Response
Move right hand left/right	End-effector $\pm Y$ velocity
Move right hand up/down	End-effector $\pm Z$ velocity
Hold W key	End-effector $+X$ (forward)
Hold S key	End-effector $-X$ (backward)
Pinch thumb+index	Gripper close signal
Release pinch	Gripper open signal
Right hand lost	All velocities $\rightarrow 0$ (safety stop)
Press q	Quit

11. Key Parameters Reference

Parameter	Location	Default	Meaning
pinch_threshold	GestureDetector	0.055	Normalized distance for pinch
deadband	HandToTwistMapper	0.06	Center dead zone
max_speed	HandToTwistMapper	0.20 m/s	Max teleop velocity
alpha	LowPassFilter	0.25	Smoothing factor
depth_speed	webcam_teleop_node	0.12 m/s	Keyboard forward/back speed
scale	servo_twist_relay_node	0.35	Input scaling for Servo
max_linear_speed	servo_twist_relay_node	0.08 m/s	Servo speed cap
max_linear_accel	servo_twist_relay_node	0.25 m/s ²	Acceleration limit
command_timeout_sec	servo_twist_relay_node	0.25 s	Deadman timeout
grasp_distance_threshold	demo_manipulation_object_node	0.16 m	Max distance to grasp

12. Data Flow Timeline (One Frame, ~33 ms @ 30 Hz)

```
t=0ms    Webcam captures BGR frame
t=1ms    Flip horizontally, convert to RGB
t=5ms    MediaPipe inference → 21 landmarks
t=6ms    GestureDetector → pinch distance
t=7ms    LowPassFilter → smoothed wrist position
t=8ms    HandToTwistMapper → (vx, vy, vz, gripper)
t=9ms    Add keyboard depth to vx
t=10ms   Publish TwistStamped + Bool to ROS topics
t=11ms   servo_twist_relay receives, scales, smooths
t=12ms   MoveIt Servo receives delta_twist_cmds
t=15ms   Jacobian IK → joint velocities
t=20ms   Panda model updates in RViz
t=33ms   Next frame
```

13. Limitations and Future Work

Current Limitation	Planned Fix
Only right hand controls robot	Dual-arm: left hand → left arm
Depth via keyboard W/S , not hand	Use hand size as depth proxy
No real physics for pick-place	Integrate Gazebo or Isaac Sim
Pinch = binary gripper	Continuous gripper force from pinch distance
Normalized coords, not metric	Hand-eye calibration for absolute positioning
Single camera	External/stereo camera for better depth
CSV recording only	Add rosbag2 recording with joint states + images

14. FAQ — Questions You Should Be Able to Answer

Q: Why velocity control instead of absolute position mapping?

A: Velocity control is more intuitive (move hand = robot moves), robust to calibration drift, and matches MoveIt Servo's primary interface. Absolute mapping would require precise hand-eye calibration.

Q: Why is the image flipped?

A: Mirror effect — when you move your hand right, the on-screen hand moves right, matching natural expectation.

Q: What happens if I lose my hand from the camera view?

A: webcam_teleop_node publishes zero velocity immediately. servo_twist_relay_node also has a 0.25 s timeout that zeros commands if messages stop.

Q: Why do we need servo_twist_relay_node? Why not connect webcam directly to Servo?

A: The relay adds scaling (webcam commands are too fast for Servo), acceleration limiting (prevents jerky motion), speed capping, smoothing, and deadman timeout — all critical for safe real robot control.

Q: What coordinate frame does the Panda use?

A: panda_link0 is the robot base. End-effector is panda_hand. MoveIt Servo expects twist commands in the frame specified in the message header.

Q: How does pinch map to gripper close?

A: Direct boolean: is_pinching = (thumb-index distance < 0.055). No hysteresis currently — rapid toggling near threshold could flicker.

Q: What's the difference between scene_objects_node and demo_manipulation_object_node?

A: scene_objects_node adds static table/cube/place-zone to the planning scene. demo_manipulation_object_node adds a dynamic object that attaches to the gripper on pinch+proximity and detaches on release.

Q: Can this data train a robot learning policy?

A: The CSV contains (observation: hand position + gesture, action: velocity + gripper). For IL you'd typically also need robot joint states, camera images, and object poses — which would require extending the recorder to subscribe to ROS topics.

Q: Why MediaPipe 0.10.14 specifically?

A: Newer MediaPipe versions removed the mp.solutions API. Version 0.10.14 is the last compatible version with the current code structure.

Q: What's the 7-day roadmap status?

A: Days 1–5 are implemented (tracking, mapping, ROS publisher, Servo connection, gripper). Day 6 (dual-arm) is partially planned (architecture supports it via separate left/right topics). Day 7 (polish/video) is ongoing.

15. Quick Command Cheat Sheet

```
# Environment (always needed)
source /opt/ros/jazzy/setup.bash
source ros2_ws/install/setup.bash
source .venv/bin/activate
export PYTHONPATH=$PWD:$PYTHONPATH

# Standalone tracking
python scripts/run_hand_tracking.py --record

# ROS teleop only
ros2 run vdat_teleop webcam_teleop_node

# Virtual EE demo
ros2 run vdat_teleop virtual_ee_simulator_node

# Full Panda demo
ros2 launch moveit_servo demo_ros_api.launch.py
ros2 service call /servo_node/switch_command_type moveit_msgs/srv/ServoCommandType "{command_type: 1}"
ros2 run vdat_teleop webcam_teleop_node
ros2 run vdat_teleop servo_twist_relay_node --ros-args -p scale:=0.35

# Debug
ros2 topic echo /teleop/right_arm/twist_cmd
ros2 topic echo /servo_node/delta_twist_cmds
ros2 topic echo /servo_node/status
```

End of Version 1 — Vision Dual-Arm Teleoperation Project Guide